

# Computation Graphs, Backpropagation and Autodiff

How gradients flow through neural networks

---

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

# Backprop vocabulary

---

# Lecture 2 built the forward map; today we reverse it

Last time, an MLP turned an input into a prediction and then one scalar loss. Today we reuse that **same graph** in reverse:

## General pattern

**Forward:** compute values

$$x \rightarrow f_{\theta}(x) = \hat{y} \rightarrow \mathcal{L}$$

**Backward:** compute sensitivities

$$\mathcal{L} \rightarrow \nabla_{\theta} \mathcal{L}$$

## Concrete scalar example

**Forward:** with  $(w, x, b, y) = (2, 3, 1, 10)$

$$\hat{y} = 2 \cdot 3 + 1 = 7, \quad \mathcal{L} = (7 - 10)^2 = 9$$

**Backward:**

$$\left( \frac{\partial \mathcal{L}}{\partial w}, \frac{\partial \mathcal{L}}{\partial b} \right) = (-18, -6)$$

**same executed graph · values flow forward, gradients flow backward**

# Backprop returns every parameter sensitivity in one sweep

Every update step is

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t).$$

A modern network has  $P = 10^7$  to  $10^{11}$  parameters.

We need all  $P$  components  $\partial \mathcal{L} / \partial \theta_j$ . Backprop computes them together in **one reverse sweep**, without perturbing parameters one at a time.

# Read the notation before the algorithm

| Symbol                                        | Meaning in this lecture                                                     |
|-----------------------------------------------|-----------------------------------------------------------------------------|
| $u, v, z$                                     | scalars                                                                     |
| $\mathbf{x}, \mathbf{z}, \boldsymbol{\theta}$ | vectors (bold lowercase)                                                    |
| $\mathbf{W}, \mathbf{X}$                      | matrices (bold uppercase)                                                   |
| $\bar{u}$                                     | $\partial \mathcal{L} / \partial u$ : the loss sensitivity carried backward |
| $\nabla_{\boldsymbol{\theta}} \mathcal{L}$    | all parameter partial derivatives stacked as a vector                       |

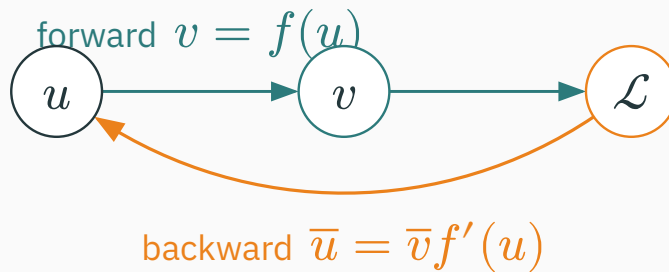
The bar is **not an average**. Read  $\bar{u}$  aloud as “the gradient arriving at  $u$ .”

# The route from one node to an entire network

| Stage               | Question we answer                                     |
|---------------------|--------------------------------------------------------|
| 1 · local rule      | How does one operation pass a gradient backward?       |
| 2 · branches        | Why do gradient contributions add?                     |
| 3 · worked graph    | Can we reproduce every derivative numerically?         |
| 4 · layers + MLP    | How do scalar rules become vector and matrix formulas? |
| 5 · autodiff + flow | How do frameworks execute and check the reverse sweep? |

# One local node

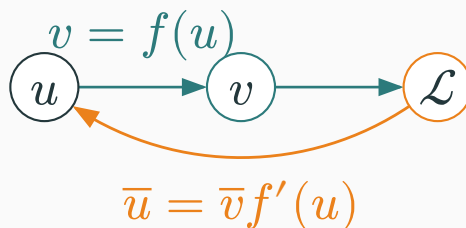
Zoom in on a single edge of the graph:  $u \rightarrow v \rightarrow \mathcal{L}$ .



**backward computes**  $\bar{u} = \bar{v} \cdot f'(u)$

# Every node multiplies upstream by its local slope

At the node  $v = f(u)$ , write the chain rule in the same bar notation as the graph:



$$\bar{u} = \bar{v} \cdot f'(u)$$

**Sent back to  $u$**

$$\bar{u} = \frac{\partial \mathcal{L}}{\partial u}$$

**Arriving at  $v$**

$$\bar{v} = \frac{\partial \mathcal{L}}{\partial v}$$

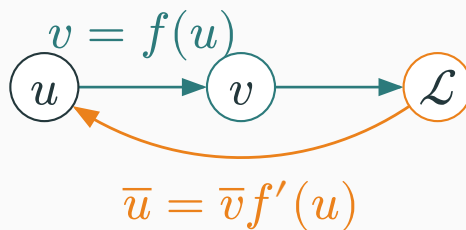
**Local slope**

$$f'(u) = \frac{\partial v}{\partial u}$$

**gradient sent backward = gradient arriving  $\times$  local slope**

# Bar notation turns the chain rule into one reusable rule

Write  $\bar{u} = \partial \mathcal{L} / \partial u$ . Then every node obeys:

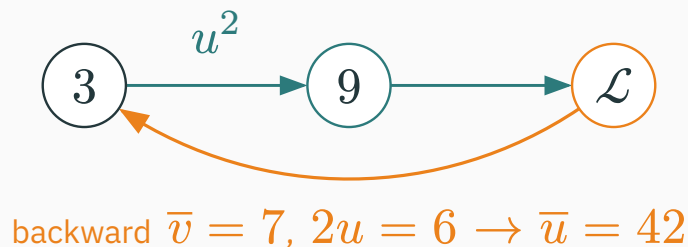


$$\bar{u} = \bar{v} \cdot \frac{\partial v}{\partial u}.$$

The node only needs to know its **own** local derivative  $\partial v / \partial u$ . It multiplies whatever gradient  $\bar{v}$  arrives from above — no global bookkeeping.

# Example: a square node

For  $v = u^2$  at  $u = 3$ , the forward value is  $v = 9$  and the local slope is  $\partial v / \partial u = 2u = 6$ .

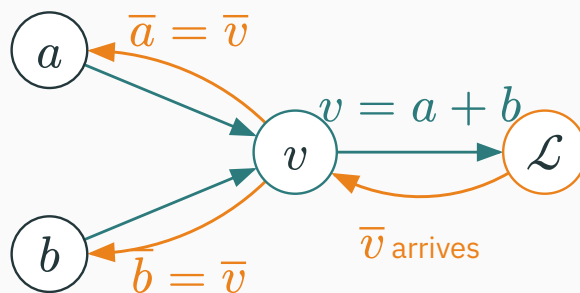


$$\bar{u} = \bar{v} \cdot 2u = 7 \cdot 6 = 42.$$

**downstream**  $\bar{u} = 42$

The node did **one** thing: multiply the incoming  $\bar{v} = 7$  by its own local slope  $2u = 6$ .

Node  $v = a + b$ . Local derivatives  $\partial v / \partial a = 1$ ,  $\partial v / \partial b = 1$ .

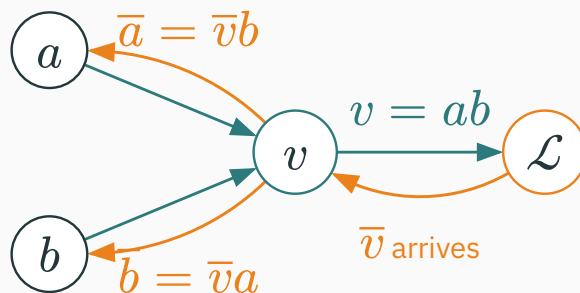


$$\bar{a} = \bar{v} \cdot 1 = \bar{v}, \quad \bar{b} = \bar{v} \cdot 1 = \bar{v}.$$

+ **copies** the upstream gradient to every input

# Multiplication sends each input the other input

Node  $v = ab$ . Local derivatives  $\partial v / \partial a = b$ ,  $\partial v / \partial b = a$ .

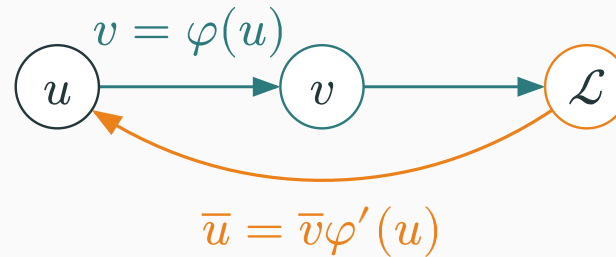


$$\bar{a} = \bar{v} \cdot b, \quad \bar{b} = \bar{v} \cdot a.$$

$\times$  sends the other input

If  $a = 2, b = 5, \bar{v} = 3$ : then  $\bar{a} = 3 \cdot 5 = 15$  and  $\bar{b} = 3 \cdot 2 = 6$ .

Node  $v = \varphi(u)$  (elementwise nonlinearity). Local derivative  $\partial v / \partial u = \varphi'(u)$ .



$$\bar{u} = \bar{v} \cdot \varphi'(u).$$

**sigmoid**  $\sigma$ :  $\varphi'(u) = \sigma(u)(1 - \sigma(u))$

**ReLU**:  $\varphi'(u) = \mathbb{1}[u > 0]$

# The rule table — memorize this

| Forward op       | Local grad    | Backward update                            |
|------------------|---------------|--------------------------------------------|
| $v = a + b$      | $1, 1$        | $\bar{a} += \bar{v}, \bar{b} += \bar{v}$   |
| $v = ab$         | $b, a$        | $\bar{a} += \bar{v}b, \bar{b} += \bar{v}a$ |
| $v = u^2$        | $2u$          | $\bar{u} += \bar{v} \cdot 2u$              |
| $v = \varphi(u)$ | $\varphi'(u)$ | $\bar{u} += \bar{v}\varphi'(u)$            |

Use +=, not = — a variable feeding several nodes accumulates gradient from **all** of them.

# **Gradients accumulate at branches**

---

# A branch must add the contribution from every path

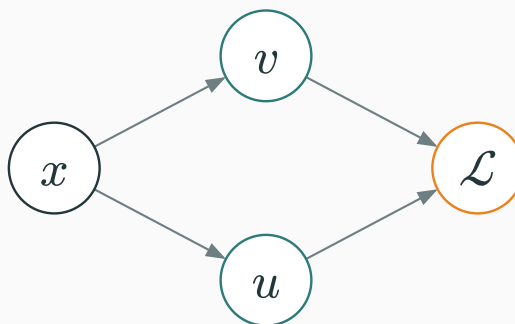
If  $x$  influences  $\mathcal{L}$  through several intermediate variables  $u_1, \dots, u_k$ :

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{i=1}^k \frac{\partial \mathcal{L}}{\partial u_i} \frac{\partial u_i}{\partial x}.$$

**gradients from every path add**

# Why gradients add at a branch

Imagine nudging  $x$  from 4 to 4.01. The **same nudge** travels down both routes to the loss:



**Via  $u = x^2$ :** the local slope is  $2x = 8$ , so  $\Delta x = 0.01$  changes the loss by about  $8 \cdot 0.01 = 0.08$ .

**Via  $v = 3x$ :** the local slope is 3, so the same nudge changes the loss by about  $3 \cdot 0.01 = 0.03$ .

$$\text{total slope} = \text{total loss change} \div \text{nudge} = \frac{0.08+0.03}{0.01} = 8 + 3 = 11$$

# Worked branch example

Let  $u = x^2$ ,  $v = 3x$ ,  $\mathcal{L} = u + v = x^2 + 3x$ . By direct calculus:

$$\frac{d\mathcal{L}}{dx} = 2x + 3, \quad \text{at } x = 4: \quad 2(4) + 3 = 11.$$

Now let us get the **same** answer by backprop.

**Forward** at  $x = 4$ :  $u = 16$ ,  $v = 12$ ,  $\mathcal{L} = 28$ . **Backward:** seed  $\overline{\mathcal{L}} = 1$ ; add node gives  $\overline{u} = 1, \overline{v} = 1$ .

$$\overline{x}_{\text{via } u} = \overline{u} \cdot 2x = 1 \cdot 8 = 8, \quad \overline{x}_{\text{via } v} = \overline{v} \cdot 3 = 3.$$

$$\overline{x} = 8 + 3 = 11. \quad \checkmark$$

# PyTorch adds the same two paths

I INTERACTIVE

```
import torch

x = torch.tensor(4.0, requires_grad=True)
u = x**2          # path 1
v = 3*x          # path 2
L = u + v
L.backward()

print(x.grad)      # tensor(11.)
```

**via  $u$ :**  $\partial \frac{u}{\partial} x = 2x = 8$

**via  $v$ :**  $\partial \frac{v}{\partial} x = 3$

**autograd accumulates both contributions:  $8 + 3 = 11$**

## `+=` is the chain rule at a branch

Both paths write into the **same**  $\bar{x}$ . If the second overwrote the first, we would lose the 8.

In PyTorch, `.grad` **accumulates** across `backward()` calls — that is exactly this rule. You must `optimizer.zero_grad()` each step, or gradients from old steps pile up.

**Why is a branch node's backward update written with += rather than =?**

**A.** It makes the forward pass faster

**B.** The variable receives a contribution from every downstream path

**C.** It changes multiplication into addition

**D.** Autodiff cannot store scalar values

**Correct: B** — The variable receives a contribution from every downstream path

**Why:** The chain rule sums all paths from the variable to the loss; assignment would discard an earlier contribution.

# **The central worked example**

---

Predict  $\hat{y} = wx + b$ , squared-error loss  $\mathcal{L} = (\hat{y} - y)^2$ . Concrete numbers:

$x = 3$ ,  $w = 2$ ,  $b = 1$ ,  $y = 10$ .

Four scalars, one loss. We will get  $\partial\mathcal{L}$  w.r.t. **all four** by hand, then check with PyTorch.

# Break into primitives

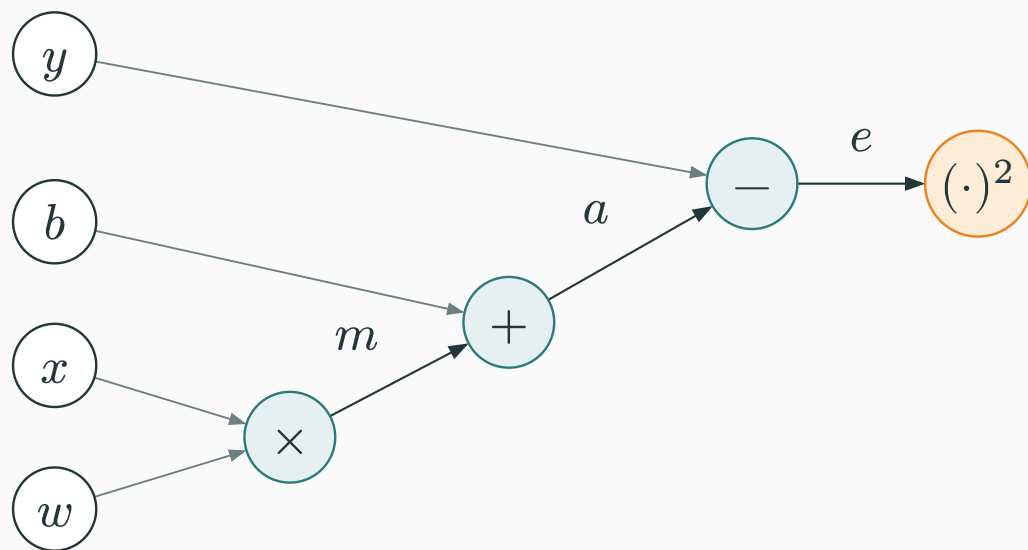
Decompose the loss into elementary nodes:

$$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$$

**$\times$ , then  $+$ , then  $-$ , then  $(\cdot)^2$**

Each is one of the nodes from our rule table.

# The computation graph



inputs (ink)  $\rightarrow$  ops  $\times, +, -$  (teal)  $\rightarrow$  loss  $(\cdot)^2$  (orange)

| Node                | Rule        | Value |
|---------------------|-------------|-------|
| $m = wx$            | $2 \cdot 3$ | 6     |
| $a = m + b$         | $6 + 1$     | 7     |
| $e = a - y$         | $7 - 10$    | -3    |
| $\mathcal{L} = e^2$ | $(-3)^2$    | 9     |

$\mathcal{L} = 9$  — now run the graph **backward**

Seed the output:  $\overline{\mathcal{L}} = \partial \mathcal{L} / \partial \mathcal{L} = 1$ . Square node  $\mathcal{L} = e^2$ , local  $\partial \mathcal{L} / \partial e = 2e$ :

$$\bar{e} = \overline{\mathcal{L}} \cdot 2e = 1 \cdot 2(-3) = -6.$$

Node  $e = a - y$ , locals  $\partial e / \partial a = 1$ ,  $\partial e / \partial y = -1$ :

$$\bar{a} = \bar{e} \cdot 1 = -6$$

$$\bar{y} = \bar{e} \cdot (-1) = 6$$

The target  $y$  already has a gradient — we would use  $\bar{y}$  if  $y$  were itself a learned quantity.

Node  $a = m + b$  **copies** its gradient:

$$\bar{m} = \bar{a} = -6, \quad \bar{b} = \bar{a} = -6.$$

So  $\bar{b} = -6$  is one of our final answers.

Node  $m = wx$  **sends the other input:**

$$\bar{w} = \bar{m} \cdot x = -6 \cdot 3 = -18$$

$$\bar{x} = \bar{m} \cdot w = -6 \cdot 2 = -12$$

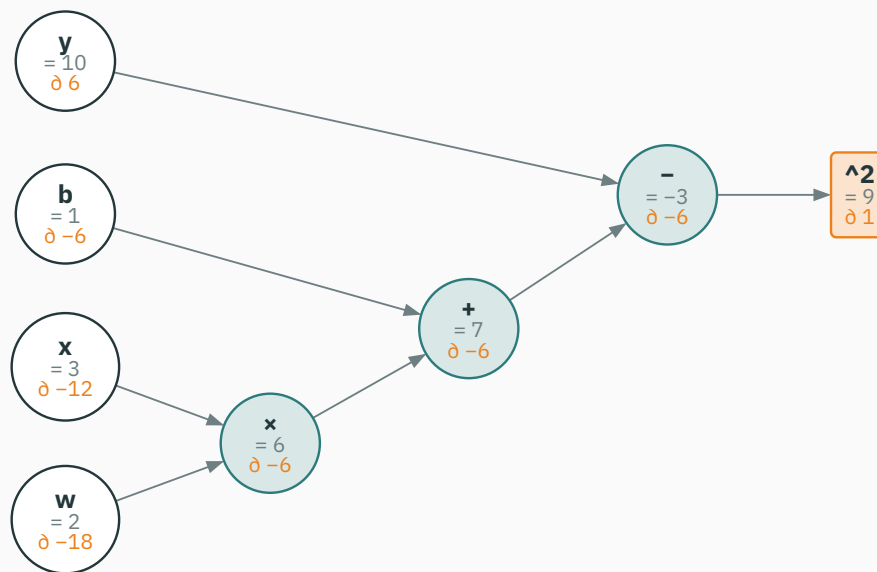
$$\bar{w} = -18, \quad \bar{x} = -12$$

| $\partial \mathcal{L} / \partial w$ | $\partial \mathcal{L} / \partial b$ | $\partial \mathcal{L} / \partial x$ | $\partial \mathcal{L} / \partial y$ |
|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| -18                                 | -6                                  | -12                                 | 6                                   |

One backward sweep produced **all four** partial derivatives — no formula re-derivation per parameter.

# The backward pass, visualized

Each node shows its forward value and its backward adjoint, **drawn by our autodiff from the real computation** — nothing typed by hand:



$\bar{w} = -18, \bar{x} = -12, \bar{b} = -6, \bar{y} = 6$  — one reverse sweep, right to left

Differentiate  $\mathcal{L} = (wx + b - y)^2$  directly. With  $wx + b - y = -3$ :

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

$$\frac{\partial \mathcal{L}}{\partial b} = 2(wx + b - y) = 2(-3) = -6 \quad \checkmark$$

**backprop = calculus — but mechanical & reusable**

| Route             | Mechanism                                           | $\partial \frac{\mathcal{L}}{\partial} w$ | Best use               |
|-------------------|-----------------------------------------------------|-------------------------------------------|------------------------|
| symbolic calculus | rewrite and differentiate the expression            | $-18$                                     | small algebra; insight |
| finite difference | perturb $w$ by $\pm \varepsilon$ and compare losses | $\approx -18$                             | debugging a gradient   |
| reverse-mode AD   | replay local rules on the executed graph            | $-18$                                     | training large models  |

**all target the same mathematical derivative; only the computation changes**

**We found  $\partial \mathcal{L} / \partial w = -18$ . For a small positive learning rate, which way does gradient descent move  $w$ ?**

**A.** Decrease  $w$

**B.** Increase  $w$

**C.** Leave  $w$  unchanged

**D.** The sign tells us nothing

Answer: a negative gradient means increase the parameter

A ANSWER

**Correct: B — Increase  $w$**

**Why:** Gradient descent subtracts the gradient:  $w \leftarrow w - \eta(-18) = w + 18\eta$ .

# A small gradient step reduces this loss

Take  $\eta = 0.01$  and update:

$$w \leftarrow 2 - 0.01(-18) = 2.18, \quad b \leftarrow 1 - 0.01(-6) = 1.06.$$

New prediction:  $\hat{y} = 2.18 \cdot 3 + 1.06 = 7.60$ .

**loss falls from 9 to  $(7.60 - 10)^2 = 5.76$**

Backprop supplied the direction. The learning rate controls how far we move; a later optimization lecture studies that choice.

## INTERACTIVE

↗ [nipunbatra.github.io/interactive-articles/autograd](https://nipunbatra.github.io/interactive-articles/autograd)

Build a scalar computation graph, run the forward pass, then click **backward** to watch each adjoint  $\bar{u}$  fill in — exactly the  $(wx + b - y)^2$  example above.

Trace which local rule changes when the final squared-error operation is replaced with  $|e|$ .

# Practice the scalar graph three ways

**I** INTERACTIVE

## See it move

Step through values and adjoints in the interactive graph ↗.

## Build it by hand

Run the manual-backprop notebook ↓.

## Let PyTorch build it

Run the autograd notebook ↓.

|               |                                                                  |
|---------------|------------------------------------------------------------------|
| <b>Easy</b>   | change one input and recompute the forward pass                  |
| <b>Medium</b> | derive all four gradients before calling <code>backward()</code> |
| <b>Hard</b>   | replace the square by $ e $ ; state what happens at $e = 0$      |

# From scalar graph to a neuron

---

# A single neuron

Vector input, one output:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad a = \varphi(z), \quad \mathcal{L} = \ell(a, y).$$

**same three nodes: affine → activation → loss**

Let  $\delta := \bar{z} = \partial \mathcal{L} / \partial z$ . Through  $a = \varphi(z)$ :

$$\delta = \bar{a} \cdot \varphi'(z), \quad \text{where } \bar{a} = \frac{\partial \mathcal{L}}{\partial a}.$$

$\delta$  is the **error signal at the pre-activation**. Everything below the neuron is expressed through this one scalar.

Through  $z = \boldsymbol{w}^\top \boldsymbol{x} + b$  with  $\bar{z} = \delta$ :

$$\nabla_{\boldsymbol{w}} \mathcal{L} = \delta \boldsymbol{x}, \quad \frac{\partial \mathcal{L}}{\partial b} = \delta, \quad \bar{\boldsymbol{x}} = \delta \boldsymbol{w}.$$

Weight gradient = error  $\delta$  times the input  $\boldsymbol{x}$ . This “error times input” shape reappears at every layer.

Sigmoid  $a = \sigma(z)$ , binary cross-entropy  $\mathcal{L} = -[y \log a + (1 - y) \log(1 - a)]$ .

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1 - y}{1 - a}$$

$$\frac{\partial a}{\partial z} = a(1 - a)$$

Multiply — the denominators cancel:

$$\bar{z} = \frac{\partial \mathcal{L}}{\partial a} \cdot a(1 - a) = a - y.$$

$$\bar{z} = a - y \text{ — prediction minus target}$$

For multi-class,  $p = \text{softmax}(z)$  and  $\mathcal{L} = -\log p_y$ :

$$\nabla_z \mathcal{L} = p - y,$$

where  $y$  is the one-hot target vector.

$$\bar{z} = p - y \text{ — the identical “prediction – target” form}$$

Choosing the loss to **match** the output nonlinearity makes the output-layer gradient trivially simple.

# The dense layer

---

A dense layer maps  $x \in \mathbb{R}^n$  to  $z \in \mathbb{R}^m$ :

$$z = Wx + b, \quad W \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m.$$

Read the shapes off the equation:  $W$  has one row per output and one column per input.

# Dense-layer backward is outer product + transpose

Given upstream  $\bar{z} \in \mathbb{R}^m$ :

$$\nabla_W \mathcal{L} = \bar{z} \mathbf{x}^\top \quad (m \times n),$$

$$\nabla_b \mathcal{L} = \bar{z} \quad (m), \quad \bar{\mathbf{x}} = \mathbf{W}^\top \bar{z} \quad (n).$$

$$\nabla_W \mathcal{L} = \bar{z} \mathbf{x}^\top \cdot \bar{\mathbf{x}} = \mathbf{W}^\top \bar{z}$$

# A $2 \times 2$ dense layer makes the forward shapes concrete

Let

$$\mathbf{x} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad \mathbf{W} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Then

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} = \begin{pmatrix} 1 & 3 \\ -2 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -4 \end{pmatrix}.$$

Two inputs, two outputs:  $\mathbf{W}$  and  $\nabla_{\mathbf{W}}\mathcal{L}$  must both be  $2 \times 2$ .

# The same $2 \times 2$ layer backward, numerically

Suppose the loss sends  $\bar{z} = \begin{pmatrix} 4 \\ -2 \end{pmatrix}$  backward.

$$\nabla_{\mathbf{W}} \mathcal{L} = \bar{z} \mathbf{x}^\top = \begin{pmatrix} 4 \\ -2 \end{pmatrix} (2 \ -1) = \begin{pmatrix} 8 & -4 \\ -4 & 2 \end{pmatrix},$$

$$\nabla_{\mathbf{b}} \mathcal{L} = \begin{pmatrix} 4 \\ -2 \end{pmatrix}, \quad \bar{\mathbf{x}} = \mathbf{W}^\top \bar{z} = \begin{pmatrix} 8 \\ 10 \end{pmatrix}.$$

**outer product for parameters · transpose for inputs**

# PyTorch verifies the dense-layer formulas

```
import torch

x = torch.tensor([2., -1.], requires_grad=True)
W = torch.tensor([[1., 3.], [-2., 1.]], requires_grad=True)
b = torch.tensor([0., 1.], requires_grad=True)

z = W @ x + b
z.backward(torch.tensor([4., -2.])) # upstream z-bar

print(W.grad) # [[ 8., -4.], [-4.,  2.]]
print(b.grad) # [4., -2.]
print(x.grad) # [8., 10.]
```

**the outer product, bias gradient, and transposed input gradient all match**

Stack  $N$  examples as rows:  $\mathbf{X} \in \mathbb{R}^{N \times n}$ ,  $\mathbf{Z} = \mathbf{X}\mathbf{W}^\top + \mathbf{1}\mathbf{b}^\top$ . Given  $\overline{\mathbf{Z}} \in \mathbb{R}^{N \times m}$ :

$$\nabla_{\mathbf{W}} \mathcal{L} = \overline{\mathbf{Z}}^\top \mathbf{X}, \quad \nabla_{\mathbf{b}} \mathcal{L} = \sum_{i=1}^N \overline{\mathbf{Z}}_i.$$

Summing rows of  $\overline{\mathbf{Z}}$  for  $\nabla_{\mathbf{b}}$  is the branch rule:  $\mathbf{b}$  is shared by all  $N$  examples, so its contributions accumulate.

| Quantity | Shape        | Must match             |
|----------|--------------|------------------------|
| $W$      | $m \times n$ | $\nabla_W \mathcal{L}$ |
| $b$      | $m$          | $\nabla_b \mathcal{L}$ |
| $x$      | $n$          | $\bar{x}$              |
| $z$      | $m$          | $\bar{z}$              |

**gradient shape = parameter shape — always**

# Backprop through a two-layer MLP

---

$$\mathbf{z}^{(1)} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \quad \mathbf{h} = \varphi(\mathbf{z}^{(1)}),$$

$$\mathbf{z}^{(2)} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2, \quad \mathbf{p} = \text{softmax}(\mathbf{z}^{(2)}), \quad \mathcal{L} = -\log p_y.$$

**affine → activation → affine → softmax → CE**

# The forward graph



backprop reverses every arrow, right to left

Softmax + CE gives the clean output-layer signal:

$$\bar{\mathbf{z}}^{(2)} = \mathbf{p} - \mathbf{y}.$$

Then, using the dense-layer rules:

$$\nabla_{\mathbf{W}_2} \mathcal{L} = \bar{\mathbf{z}}^{(2)} \mathbf{h}^\top, \quad \bar{\mathbf{h}} = \mathbf{W}_2^\top \bar{\mathbf{z}}^{(2)}.$$

Through the activation  $\mathbf{h} = \varphi(\mathbf{z}^{(1)})$  (elementwise):

$$\bar{\mathbf{z}}^{(1)} = \bar{\mathbf{h}} \odot \varphi'(\mathbf{z}^{(1)}).$$

Then the first affine layer:

$$\nabla_{\mathbf{W}_1} \mathcal{L} = \bar{\mathbf{z}}^{(1)} \mathbf{x}^\top, \quad \nabla_{\mathbf{b}_1} \mathcal{L} = \bar{\mathbf{z}}^{(1)}.$$

$\odot$  is elementwise (Hadamard). Each activation coordinate depends only on its matching pre-activation, so the Jacobian is diagonal.

| Forward                       | Backward                                          |
|-------------------------------|---------------------------------------------------|
| $z^{(1)} = W_1 x + b_1$       | $\bar{z}^{(1)} = \bar{h} \odot \varphi'(z^{(1)})$ |
| $h = \varphi(z^{(1)})$        | $\nabla_{W_1} = \bar{z}^{(1)} x^\top$             |
| $z^{(2)} = W_2 h + b_2$       | $\bar{h} = W_2^\top \bar{z}^{(2)}$                |
| $p = \text{softmax}(z^{(2)})$ | $\nabla_{W_2} = \bar{z}^{(2)} h^\top$             |
| $\mathcal{L} = -\log p_y$     | $\bar{z}^{(2)} = p - y$                           |

**same two moves at every layer:  $\delta x^\top$  for weights,  $W^\top \delta$  for inputs**

## INTERACTIVE

↗ [nipunbatra.github.io/interactive-articles/autograd](https://nipunbatra.github.io/interactive-articles/autograd)

Step through a small MLP's forward pass, then watch  $\bar{z}^{(2)} = p - y$  propagate back through  $W_2$ , the activation, and  $W_1$  — the adjoints light up layer by layer.

Trace the same upstream adjoint into the parameter and input branches of the affine layer.

# A tiny MLP uses the same recipe

```
import torch
from torch import nn
import torch.nn.functional as F

model = nn.Sequential(
    nn.Linear(2, 3), nn.ReLU(), nn.Linear(3, 2)
)
x = torch.tensor([[2., -1.]])
target = torch.tensor([1])

loss = F.cross_entropy(model(x), target)
loss.backward()
for name, p in model.named_parameters():
    print(name, tuple(p.grad.shape))
```

```
0.weight (3, 2) · 0.bias (3,) · 2.weight (2, 3) · 2.bias (2,)
```

**each parameter receives a gradient of exactly the same shape**

**For  $z = Wx + b$ , which pair uses the same upstream adjoint  $\bar{z}$ ?**

**A.**  $\nabla_W = \bar{z}x^\top$  and  $\bar{x} = W^\top \bar{z}$

**B.**  $\nabla_W = W^\top x$  and  $\bar{x} = \bar{z}x^\top$

**C.**  $\nabla_W = x$  and  $\bar{x} = W$

**D.**  $\nabla_W = 0$  and  $\bar{x} = 0$

**Correct: A** —  $\nabla_{\mathbf{W}} = \bar{\mathbf{z}}\mathbf{x}^\top$  and  $\bar{\mathbf{x}} = \mathbf{W}^\top \bar{\mathbf{z}}$

**Why:** Both branches apply the chain rule to the same output adjoint; they differ only in the local derivative of the affine operation.

# Automatic differentiation

---

# Backprop is reverse-mode automatic differentiation

All three routes answer the same question, but they do different work:

**Symbolic** rewrites a mathematical expression into a derivative expression.

**Finite difference** probes nearby inputs and estimates a derivative numerically.

**Reverse-mode AD** replays exact local rules on the operations that actually ran.

**backprop is reverse-mode AD specialized to neural-network computation graphs**

# A scalar loss makes reverse mode the right direction

A loss is a map  $f : \mathbb{R}^P \rightarrow \mathbb{R}$ : many parameter inputs, one output.

**Forward mode:** to obtain the full gradient, run one directional sweep per parameter  $\rightarrow P$  sweeps.

**Reverse mode:** one reverse sweep per scalar output  $\rightarrow$  one sweep for  $\mathcal{L}$ .

**one reverse sweep costs on the same order as a small number of forward evaluations**

# Autodiff records values forward and replays rules backward

During the forward pass the framework:

- **records** each primitive op and its inputs onto a tape (the graph);
- **stores** intermediate values (needed by local derivatives);
- on `backward()`: seeds  $\overline{\mathcal{L}} = 1$ , applies each op's local rule, and **accumulates** into `.grad`.

**you write only the forward pass — the backward pass is generated**

```
import torch
x = torch.tensor(3.0)
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(10.0)

L = (w*x + b - y)**2      # forward:  L = 9
L.backward()              # reverse-mode autodiff

print(L.item())           # 9.0
print(w.grad, b.grad)     # tensor(-18.)  tensor(-6.)
```

**matches our hand computation exactly**

Our own toolkit ships a tiny reverse-mode autodiff (chalkdust autodiff) — build the expression, get **every** gradient in one backward pass. This slide runs it:

```
#import "@local/chalkdust-autodiff:0.1.0" as ad
#let L = ad.expr("(w*x + b - y)^2", ("w", "x", "b", "y"))
#ad.value(L, (2, 3, 1, 10)) // the loss
#ad.grad(L, (2, 3, 1, 10)) // ( $\bar{w}$ ,  $\bar{x}$ ,  $\bar{b}$ ,  $\bar{y}$ ), one backward pass
```

**computed live in this slide:  $\mathcal{L} = 9$ ,  $\bar{w} = -18$ ,  $\bar{x} = -12$ ,  $\bar{b} = -6$ ,  $\bar{y} = 6$  — the same four numbers, by hand, by PyTorch, and by our own graph.**

# Gradients accumulate in frameworks

.grad is **added to**, never reset, by backward(). The standard loop is:

```
for x, y in loader:
    optimizer.zero_grad()      # clear old gradients
    loss = criterion(model(x), y)
    loss.backward()           # accumulate this batch's gradients
    optimizer.step()          # theta <- theta - eta * grad
```

Forget zero\_grad() and you sum gradients across steps — the same += branch rule, now biting you.

# Gradient checking & gradient flow

---

To **debug** an analytic gradient, compare against a central difference:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} \approx \frac{\mathcal{L}(\boldsymbol{\theta} + \varepsilon \mathbf{e}_j) - \mathcal{L}(\boldsymbol{\theta} - \varepsilon \mathbf{e}_j)}{2\varepsilon}.$$

Useful in small float64 unit tests; far too slow for training — a central difference needs **two** loss evaluations per parameter.

# Gradient check: finite difference versus PyTorch

```
import torch

def loss(w):
    return (w*3.0 + 1.0 - 10.0)**2

w, eps = 2.0, 1e-5
g_fd = (loss(w + eps) - loss(w - eps)) / (2*eps)

wt = torch.tensor(w, dtype=torch.float64, requires_grad=True)
loss(wt).backward()
g_ad = wt.grad.item()

print(g_fd, g_ad)          # both approximately -18
print(abs(g_fd - g_ad))   # numerical error near zero
```

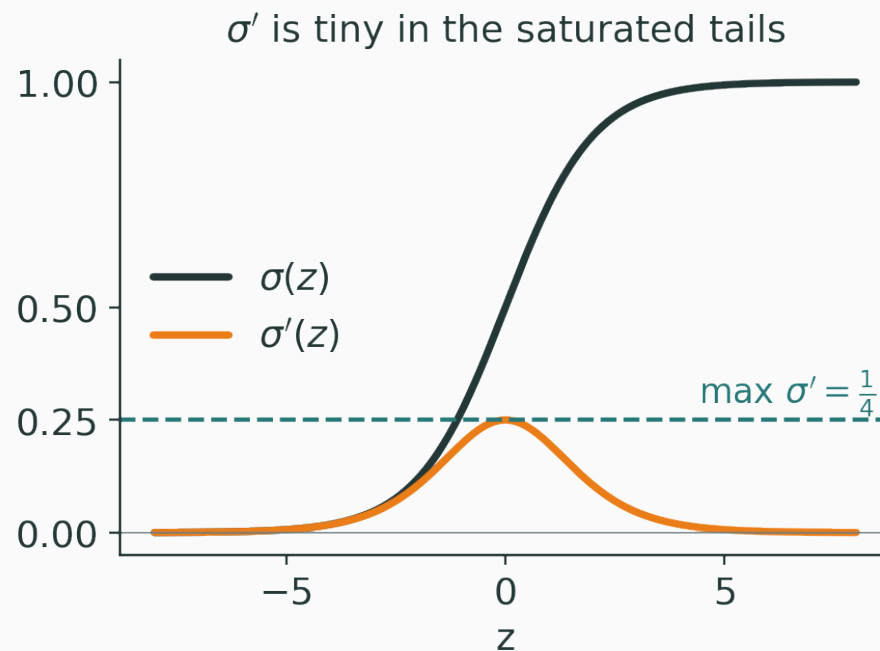
Use float64. Too-large  $\varepsilon$  gives truncation error; too-small  $\varepsilon$  gives cancellation error. Check a few coordinates—not millions.

Stack  $L$  layers  $h_0 \rightarrow h_1 \rightarrow \dots \rightarrow h_L$ . The chain rule multiplies Jacobians:

$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{\ell=1}^L \frac{\partial h_\ell}{\partial h_{\ell-1}}.$$

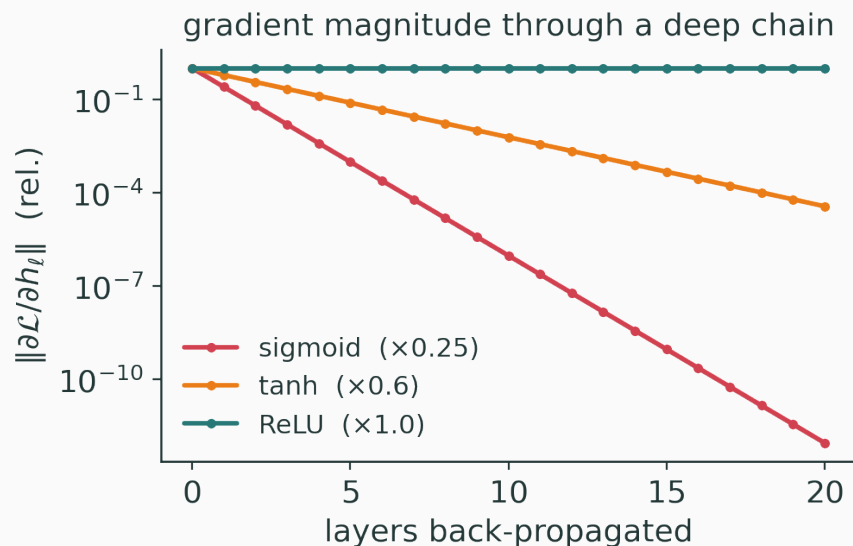
**a product of  $L$  terms — it can vanish or explode**

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$ , and  $\approx 0$  for large  $|z|$ :



each sigmoid layer multiplies the gradient by at most  $1/4$

Multiply many small local derivatives  $\rightarrow$  the signal **decays** with depth:



Sigmoid derivatives can repeatedly shrink the signal. ReLU has slope 1 on active units, but slope 0 on inactive units; weights and normalization also affect the full Jacobian.

**Along a scalar chain, ten consecutive local derivatives equal 0.5. What factor reaches the earliest node?**

**A.** 5

**B.** 0.5

**C.**  $0.5^{10} \approx 0.001$

**D.**  $10^{0.5}$

**Correct: C** —  $0.5^{10} \approx 0.001$

**Why:** Backprop multiplies local derivatives along a path; several individually moderate slopes can yield a tiny product.

## INTERACTIVE

↗ [nipunbatra.github.io/interactive-articles/vanishing-gradients](https://nipunbatra.github.io/interactive-articles/vanishing-gradients)

Sweep depth and activation function; watch the gradient magnitude at layer 0 collapse for sigmoid and survive for ReLU.

Sweep depth to see repeated small local slopes rapidly erase an early-layer gradient.

| Level  | Problem                                           | Verification                                                  |
|--------|---------------------------------------------------|---------------------------------------------------------------|
| easy   | $v = (a + b)^2$ at $a = 2, b = -1$                | derive $\bar{a}, \bar{b}$ ; check with PyTorch                |
| medium | $\mathcal{L} = x^2 + 3x$ at $x = 4$               | show the two path contributions sum to 11                     |
| hard   | $\mathcal{L} = \ell(\mathbf{W}x + \mathbf{b}, y)$ | check one entry of $\nabla_{\mathbf{W}}$ by finite difference |

**solve by hand first · verify the whole gradient with autograd · spot-check numerically**

# Summary

---

# Backprop in one sentence

Seed  $\bar{\mathcal{L}} = 1$ , then walk the graph **backward**, at each node computing

$$\text{downstream} = \text{upstream} \times \text{local},$$

and **adding** gradients wherever a variable branched.

For vector nodes, “times local” becomes a vector–Jacobian product; the logic is unchanged.

| Setting      | Backward rule                                                                                                       |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| scalar node  | $\bar{u} = \bar{v} \cdot \partial v / \partial u$                                                                   |
| dense layer  | $\nabla_{\mathbf{W}} = \bar{\mathbf{z}} \mathbf{x}^\top, \quad \bar{\mathbf{x}} = \mathbf{W}^\top \bar{\mathbf{z}}$ |
| activation   | $\bar{\mathbf{z}} = \bar{\mathbf{a}} \odot \varphi'(z)$                                                             |
| softmax + CE | $\bar{\mathbf{z}} = \mathbf{p} - \mathbf{y}$                                                                        |
| branch point | gradients <b>add</b>                                                                                                |

# What comes next

We can now compute  $\nabla_{\theta} \mathcal{L}$  for any network by composing local rules.

**Next (Lecture 3A): the geometry behind these objects – gradients, Jacobians, and Hessians**

Today used derivatives operationally. Lecture 3A makes their shapes and geometry explicit; the later optimization lecture then uses the gradient to choose parameter steps.

We can compute  $\nabla_{\theta} \mathcal{L}$ .

Next: what gradients, Jacobians, and Hessians mean geometrically.

Lecture 3A · Derivatives, Gradients, Jacobians & Hessians